BỘ NÔNG NGHIỆP VÀ MÔI TRƯỜNG
PHÂN HIỆU TRƯỜNG ĐẠI HỌC THỦY LỢI
BỘ MÔN: CÔNG NGHỆ THÔNG TIN



ĐỒ ÁN MÔN HỌC: QUẢN LÝ DỮ LIỆU LỚN

Tên đề tài: XÂY DỰNG LUỒNG XỬ LÝ DỮ LIỆU ĐƠN HÀNG VÀ GIAO DỊCH THƯƠNG MẠI ĐIỆN TỬ THỜI GIAN THỰC ĐA NGUỒN

(Ứng dụng Apache Airflow, Kafka, Spark Streaming và Cassandra)

GVHD: Cô Nguyễn Thị Hồng Tú

SINH VIÊN THỰC HIỆN:

1. Lê Trung Hiếu
2. Nguyễn Huỳnh Ngọc Duyên

TP HỒ CHÍ MINH - 2026

PHẦN 1: TỔNG QUAN VỀ ĐỀ TÀI VÀ KIẾN TRÚC HỆ THỐNG 2
1.1. Lý do chọn đề tài và Mục đích 3
1.2. Mô hình kiến trúc hệ thống 3
1.3. Bảng phân công vai trò các công nghệ 4
PHẦN 2: CHI TIẾT CÁC BƯỚC THỰC HIỆN 4
2.1. Thu thập và gửi dữ liệu vào Kafka 5
2.2. Điều phối tác vụ bằng Airflow 6
2.3. Xử lý dữ liệu với Spark Streaming 8
2.4. Lưu trữ vào Cassandra 10
PHẦN 3: KẾT QUẢ KIỂM THỬ VÀ ĐÁNH GIÁ 12
3.1. Kết quả kiểm thử các trường hợp dữ liệu 12
3.2. Đánh giá độ trễ và hiệu năng 13
3.3. Hạn chế 13
PHẦN 4: BẢNG THEO DÕI HỆ THỐNG TRÊN LOCALHOST 13
HƯỚNG DẪN CÁC BƯỚC CHẠY ĐỒ ÁN 14

PHẦN 1: TỔNG QUAN VỀ ĐỀ TÀI VÀ KIẾN TRÚC HỆ THỐNG

1.1. Lý do chọn đề tài và Mục đích

1. Lý do chọn đề tài: Hiện nay các trang thương mại điện tử có lượng giao dịch rất lớn phát sinh liên tục. Việc xử lý dữ liệu theo kiểu truyền thống (Batch Processing) có độ trễ cao, khó phát hiện các giao dịch bất thường kịp thời. Vì vậy nhóm chúng em chọn đề tài xây dựng đường ống xử lý dữ liệu theo thời gian thực (Real-time Data Pipeline) để tìm hiểu cách thu thập, làm sạch và lưu trữ dữ liệu liên tục

2. Mục đích nghiên cứu:

- Tìm hiểu cách kết nối các công nghệ Big Data: Airflow, Kafka, Spark Streaming và Cassandra.
- Thực hành xử lý dữ liệu lỗi (dữ liệu thiếu, số tiền âm, trùng lặp) bằng tính năng Watermark trong Spark.
- Tính toán doanh thu theo danh mục sản phẩm và cảnh báo các đơn hàng giá trị cao trên $1000 ngay khi dữ liệu đổ về.

3. Các công nghệ sử dụng trong đồ án:

Apache Airflow: Dùng để lập lịch và điều phối các tác vụ.

Apache Kafka (3 Brokers): Làm hàng chờ đệm dữ liệu, tránh mất mát dữ liệu khi hệ thống bị quá tải.

Apache Spark Streaming: Nhận dữ liệu từ Kafka, xử lý lọc rác và tính toán độ trễ.

Apache Cassandra: Cơ sở dữ liệu NoSQL dùng để lưu trữ dữ liệu sau khi đã làm sạch.

Docker Compose: Giúp đóng gói và chạy toàn bộ hệ thống gồm 11 container trên máy local.

1.2. Mô hình kiến trúc hệ thống

Luồng xử lý dữ liệu của hệ thống: dữ liệu:

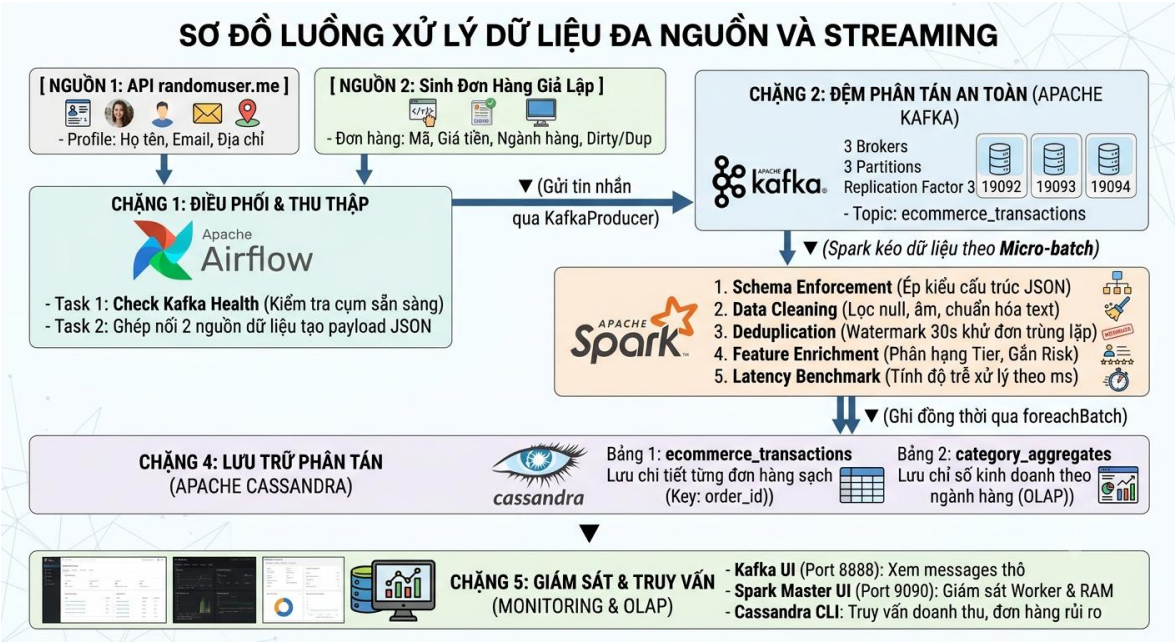
1. Lấy thông tin người dùng từ API (randomuser.me) và tự sinh các đơn hàng giả lập (bao gồm cả đơn hàng sạch và đơn hàng bị lỗi/trùng).

2. Airflow: Kiểm tra kết nối Kafka trước, sau đó kích hoạt script gửi dữ liệu.

3. Kafka: Nhận dữ liệu vào các Topic (ecommerce_transactions), phân tải qua 3 Broker.
4. Spark Streaming: Đọc dữ liệu từ Kafka, tiến hành ép kiểu JSON, lọc bỏ bản ghi rác/âm, khử trùng lặp qua Watermark 30 giây, phân loại giá trị đơn hàng và tính độ trễ.
5. Cassandra: Ghi dữ liệu vào 2 bảng (bảng chi tiết giao dịch và bảng tổng hợp doanh thu theo danh mục).

1.3. Bảng phân công vai trò các công nghệ và luồng xử lý

Công nghệ	Nhiệm vụ trong bài	Lý do sử dụng
Docker Compose	Tạo môi trường chạy 11 container	Dễ cài đặt, không lo bị lỗi khác biệt môi trường.
Apache Airflow	Chạy script thu thập dữ liệu	Tự động hóa quy trình và tự thử lại (retry) nếu gặp lỗi.
Apache Kafka	Làm bộ đệm truyền tin	Tốc độ truyền tin nhanh, có 3 broker để sao lưu dữ liệu.
Apache Spark	Xử lý và tính toán dữ liệu	Xử lý dữ liệu trực tiếp trên bộ nhớ RAM rất nhanh.
Apache Cassandra	Lưu trữ dữ liệu kết quả	Tốc độ ghi dữ liệu rất nhanh, không có điểm nghẽn.



PHẦN 2: CHI TIẾT CÁC BƯỚC THỰC HIỆN

2.1. Thu thập và gửi dữ liệu vào Kafka

Nhóm viết file ecommerce_producer.py để lấy thông tin khách hàng từ API randomuser.me kết hợp với việc tự tạo đơn hàng thuộc 5 ngành hàng (Electronics, Fashion, Home & Kitchen, Books & Media, Beauty & Health).

Để test khả năng xử lý dữ liệu của Spark, nhóm có cố tình tạo thêm một số dữ liệu lỗi như: để trống order_id, email bị null, số tiền bị âm hoặc cố tình gửi trùng order_id cũ.

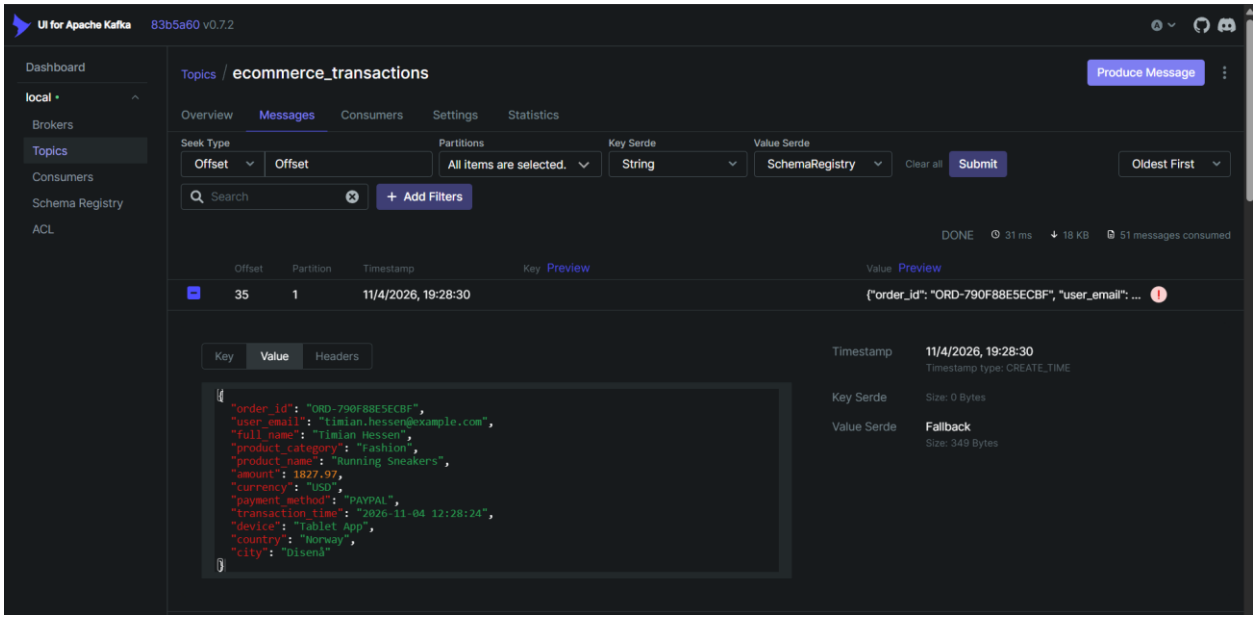
Code tạo dữ liệu giả lập :

```
# Trích từ PROJECT1/dags/ecommerce_producer.py
def generate_transaction_event(user_profile: dict, force_dirty_type: str = None) -> dict:
	category = random.choice(list(PRODUCT_CATALOG.keys()))
	product = random.choice(PRODUCT_CATALOG[category])

# 85% giá thông thường 20 - 950 USD, 15% giao dịch cao cấp 1000 - 3000 USD (Risk Flag)
	amount = round(random.uniform(1050.0, 3200.0), 2) if random.random() < 0.15 else
round(random.uniform(15.0, 950.0), 2)
	order_id = f"ORD-{uuid.uuid4().hex[:12].upper()}"
	tx_time = datetime.utcnow().strftime("%Y-%m-%d %H:%M:%S")

event = {
	"order_id": order_id, "user_email": user_profile["email"],
	"full_name": user_profile["full_name"], "product_category": category,
	"product_name": product, "amount": amount, "currency": "USD",
	"payment_method": random.choice(PAYMENT_METHODS), "transaction_time": tx_time,
	"device": random.choice(DEVICES), "country": user_profile["country"], "city":
user_profile["city"]
}
# Chèn dữ liệu lỗi/bất thường có kiểm soát
if force_dirty_type == "missing_order_id": event["order_id"] = ""
elif force_dirty_type == "missing_email": event["user_email"] = None
elif force_dirty_type == "negative_amount": event["amount"] = -round(random.uniform(10.0,
100.0), 2)
```

Hình ảnh minh chứng & Thao tác kiểm tra http://localhost:8888



2.2. Điều phối tác vụ bằng Airflow

Tạo DAG ecommerce_dag.py để quản lý việc chạy. Đầu tiên Airflow sẽ kiểm tra xem Kafka có đang hoạt động không (health check), nếu bình thường thì mới chạy task tiếp theo để gửi dữ liệu.

Code Airflow DAG :

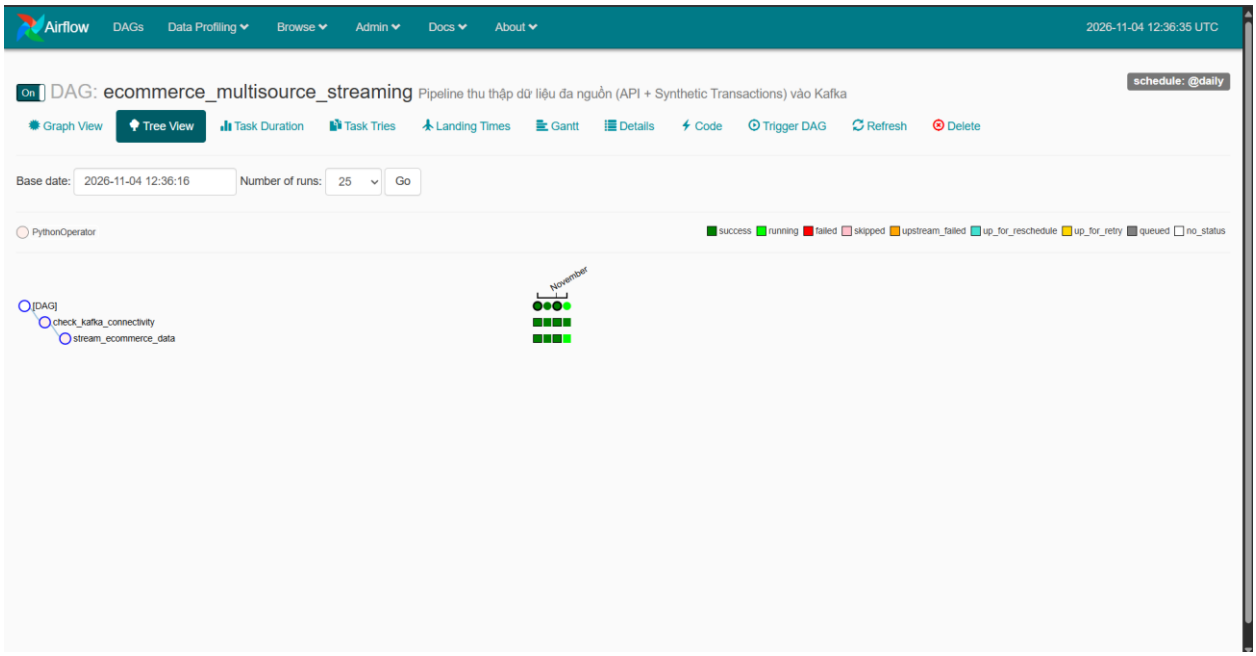
```
# Trích từ PROJECT1/dags/ecommerce_dag.py
        default_args = {
        "owner": "data_engineer_team",
        "depends_on_past": False,
        "start_date": datetime(2026, 10, 20),
        "retries": 2,
        "retry_delay": timedelta(seconds=10),
        "on_failure_callback": task_failure_alert,
}

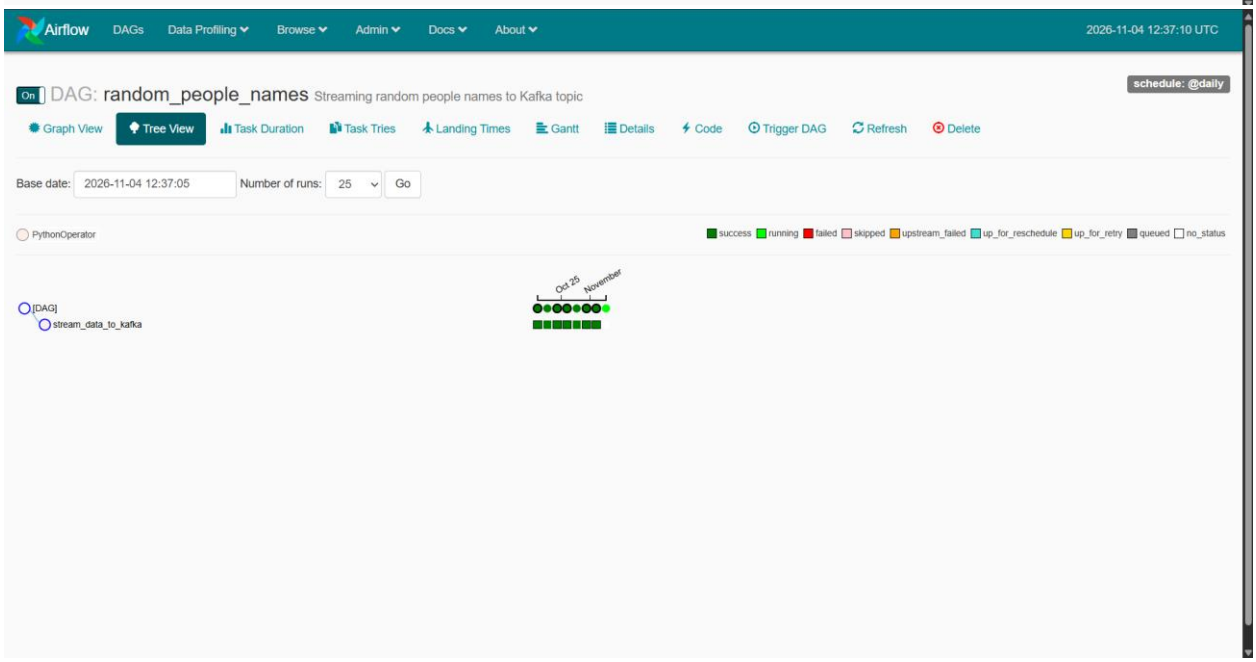
dag = DAG(
        dag_id="ecommerce_multisource_streaming",
        default_args=default_args,
        schedule_interval="@daily",
        catchup=False,
)

check_kafka_task = PythonOperator(task_id="check_kafka_connectivity",
python_callable=check_kafka_cluster_health, dag=dag)
stream_data_task = PythonOperator(task_id="stream_ecommerce_data",
python_callable=run_streaming_job, dag=dag)

# Thiết lập quan hệ phụ thuộc chặt chẽ
check_kafka_task >> stream_data_task
```

Hình ảnh minh chứng & Thao tác kiểm tra http://localhost:8080





2.3. Xử lý dữ liệu với Spark Streaming

Trong file spark_streaming_advanced.py, nhóm thực hiện các bước xử lý sau:

1. Chuyển dữ liệu Kafka dạng binary về dạng JSON theo đúng schema.
2. Lọc bỏ các bản ghi bị rác: loại bỏ order_id rỗng, email null và amount <= 0.
3. Chuẩn hóa dữ liệu: viết thường email, viết hoa chữ cái đầu cho tên thành phố và quốc gia.
4. Dùng Watermark 30 giây để xóa các đơn hàng bị trùng lặp (dropDuplicates).
5. Phân loại đơn hàng (HIGH_VALUE >= 500$, MEDIUM_VALUE >= 100$) và đánh dấu cảnh báo rủi ro nếu đơn hàng >= 1000$.

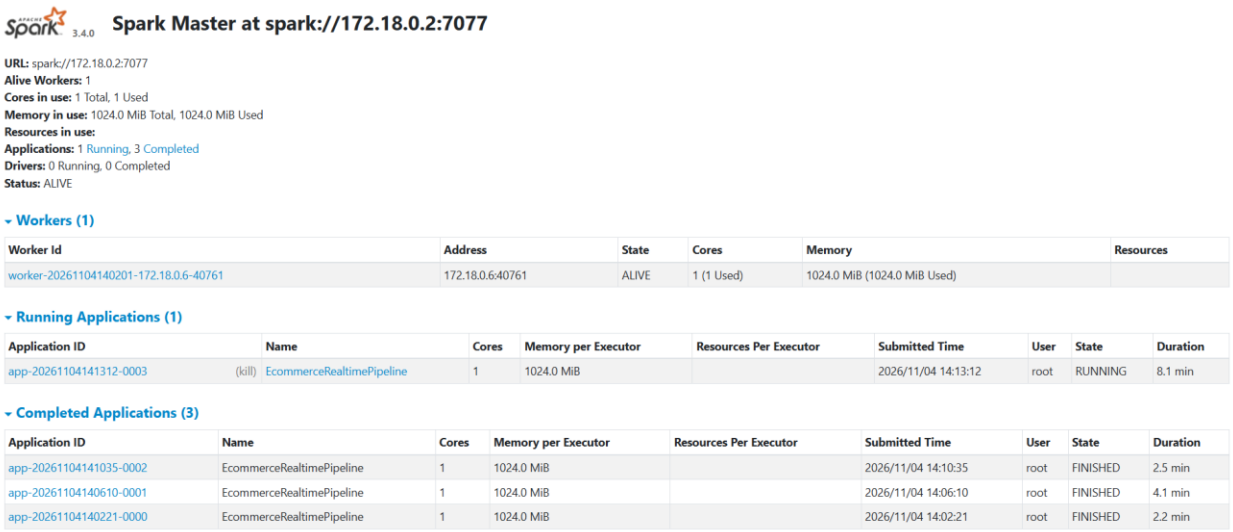
Code xử lý dữ liệu Spark :

```
# Trích từ PROJECT1/spark_streaming_advanced.py
# Làm sạch dữ liệu
cleaned_df = (
       parsed_df
       .filter(col("order_id").isNotNull() & (trim(col("order_id")) != ""))
       .filter(col("user_email").isNotNull() & (trim(col("user_email")) != ""))
       .filter(col("amount") > 0.0)
       .withColumn("user_email", trim(lower(col("user_email"))))
       .withColumn("city", initcap(trim(col("city"))))
       .withColumn("country", initcap(trim(col("country"))))
       .withColumn("transaction_time", to_timestamp(col("transaction_time"), "yyyy-MM-dd
HH:mm:ss"))
)

# Khử trùng lặp bằng Watermark
deduplicated_df = (
       cleaned_df
       .withWatermark("transaction_time", "30 seconds")
       .dropDuplicates(["order_id", "transaction_time"])
)

# Gắn nhãn phân loại & Đo lường độ trễ (ms)
enriched_df = (
       deduplicated_df
       .withColumn("tier", when(col("amount") >= 500.0, "HIGH_VALUE").when(col("amount") >=
100.0, "MEDIUM_VALUE").otherwise("LOW_VALUE"))
       .withColumn("risk_flag", when(col("amount") >= 1000.0, True).otherwise(False))
       .withColumn("processed_time", current_timestamp())
       .withColumn("processing_latency_ms", (current_timestamp().cast("long")*1000) -
(col("transaction_time").cast("long")*1000))
)
```

Hình ảnh minh chứng & Thao tác kiểm tra http://localhost:9090



Spark 3.4.0 **Spark Master at spark://172.18.0.2:7077**

URL: spark://172.18.0.2:7077
Alive Workers: 1
Cores in use: 1 Total, 1 Used
Memory in use: 1024.0 MiB Total, 1024.0 MiB Used
Resources in use:
Applications: 1 Running, 3 Completed
Drivers: 0 Running, 0 Completed
Status: ALIVE

Workers (1)

Worker Id	Address	State	Cores	Memory	Resources
worker-20261104140201-172.18.0.6-40761	172.18.0.6:40761	ALIVE	1 (1 Used)	1024.0 MiB (1024.0 MiB Used)	

Running Applications (1)

Application ID	Name	Cores	Memory per Executor	Resources Per Executor	Submitted Time	User	State	Duration
app-20261104141312-0003 (kill)	EcommerceRealtimePipeline	1	1024.0 MiB		2026/11/04 14:13:12	root	RUNNING	8.1 min

Completed Applications (3)

Application ID	Name	Cores	Memory per Executor	Resources Per Executor	Submitted Time	User	State	Duration
app-20261104141035-0002	EcommerceRealtimePipeline	1	1024.0 MiB		2026/11/04 14:10:35	root	FINISHED	2.5 min
app-20261104140610-0001	EcommerceRealtimePipeline	1	1024.0 MiB		2026/11/04 14:06:10	root	FINISHED	4.1 min
app-20261104140221-0000	EcommerceRealtimePipeline	1	1024.0 MiB		2026/11/04 14:02:21	root	FINISHED	2.2 min

2.4. Lưu trữ vào Cassandra

Tạo 2 bảng trong Keyspace spark_streaming: Bảng ecommerce_transactions: Lưu thông tin chi tiết từng đơn hàng sau khi làm sạch

Bảng category_aggregates: Lưu tổng hợp số lượng đơn và tổng doanh thu theo từng ngành hàng

Schema CQL trong Cassandra:

```
Trích từ PROJECT1/schema_advanced.cql
CREATE KEYSPACE IF NOT EXISTS spark_streaming WITH replication = {'class': 'SimpleStrategy',
'replication_factor': 1};

CREATE TABLE spark_streaming.ecommerce_transactions (
       order_id text PRIMARY KEY, user_email text, full_name text, product_category text,
       product_name text, amount double, currency text, payment_method text,
       transaction_time timestamp, device text, country text, city text,
       tier text, risk_flag boolean, processed_time timestamp, processing_latency_ms bigint
);

CREATE TABLE spark_streaming.category_aggregates (
       product_category text, window_start timestamp,
       total_orders bigint, total_revenue double, avg_order_value double, risk_count bigint,
       PRIMARY KEY ((product_category), window_start)
) WITH CLUSTERING ORDER BY (window_start ASC);
```

Hình ảnh minh chứng & Thao tác kiểm tra trong Terminal

Lệnh kiểm tra bảng Chi tiết:

docker exec cassandra cqlsh -e "SELECT order_id, full_name, user_email, product_category, amount, tier, risk_flag FROM spark_streaming.ecommerce_transactions LIMIT 10;"

```
tui@LAPTOP-VFMF2B0O:/mnt/c/Users/ADMIN/OneDrive/Documents/PROJECT1$ docker exec cassandra cqlsh -e "SELECT order_id, full_name, user_email, product_category
, amount, tier, risk_flag FROM spark_streaming.ecommerce_transactions LIMIT 10;"

 order_id         | full_name        | user_email                   | product_category | amount  | tier         | risk_flag
------------------+------------------+------------------------------+------------------+---------+--------------+-----------
 ORD-2A1182D2E469 |        Utz Kober |        utz.kober@example.com |   Home & Kitchen | 3053.76 |   HIGH_VALUE |      True
 ORD-425E5BE0DAA8 | Serafim Mandrich | serafim.mandrich@example.com |  Beauty & Health |  803.95 |   HIGH_VALUE |     False
 ORD-5FCC1C731944 |     Alexis Novak |     alexis.novak@example.com |  Beauty & Health |  301.29 | MEDIUM_VALUE |     False
 ORD-E79C10064849 |   Alexis Legrand |   alexis.legrand@example.com |    Books & Media |   92.34 |    LOW_VALUE |     False
 ORD-43ACA6242921 |       Ken Evans |       ken.evans@example.com |      Electronics |  944.89 |   HIGH_VALUE |     False
 ORD-E394B0A9EEE3 |    Esteban David |    esteban.david@example.com |   Home & Kitchen |  826.44 |   HIGH_VALUE |     False
 ORD-EE6AF48597E6 |      رظن وكن انث |      thn.nkwnzr@example.com |      Electronics |  262.02 | MEDIUM_VALUE |     False
 ORD-4708981B0D9A |    Damjan Pantić |    damjan.pantic@example.com |    Books & Media |   858.7 |   HIGH_VALUE |     False
 ORD-8B477DD3E061 |      Ella Knight |      ella.knight@example.com |   Home & Kitchen |  181.38 | MEDIUM_VALUE |     False
 ORD-DF98F2537B3A |     Samuel Olson |     samuel.olson@example.com |   Home & Kitchen |  442.26 | MEDIUM_VALUE |     False

(10 rows)
```

Lệnh kiểm tra bảng Tổng hợp Doanh thu:

docker exec cassandra cqlsh -e "SELECT product_category, total_orders, total_revenue, avg_order_value, risk_count FROM spark_streaming.category_aggregates;"

```
tui@LAPTOP-VFMF2B0O:/mnt/c/Users/ADMIN/OneDrive/Documents/PROJECT1$ docker exec cassandra cqlsh -e "SELECT product_category, total_orders, total_revenue, av
g_order_value, risk_count FROM spark_streaming.category_aggregates;"

 product_category | total_orders | total_revenue | avg_order_value | risk_count
------------------+--------------+---------------+-----------------+------------
   Home & Kitchen |            2 |       2025.64 |         1012.82 |          1
  Beauty & Health |            3 |       1388.45 |          462.82 |          0
    Books & Media |            2 |        954.54 |          477.27 |          0
      Electronics |            2 |        291.82 |          145.91 |          0
      Electronics |            1 |        944.89 |          944.89 |          0

(5 rows)
```

docker logs spark_master --tail 20

```
tui@LAPTOP-VFMF2B0O:/mnt/c/Users/ADMIN/OneDrive/Documents/PROJECT1$ docker logs spark_master --tail 20
26/11/04 15:04:01 INFO BlockManagerMaster: Registered BlockManager BlockManagerId(driver, spark-master, 43125, None)
26/11/04 15:04:01 INFO BlockManager: Initialized BlockManager: BlockManagerId(driver, spark-master, 43125, None)
26/11/04 15:04:01 INFO Master: Launching executor app-20261104150401-0000/0 on worker worker-20261104140201-172.18.0.6-40761
26/11/04 15:04:01 INFO StandaloneAppClient$ClientEndpoint: Executor added: app-20261104150401-0000/0 on worker-20261104140201-172.18.0.6-40761 (172.18.0.6:4
0761) with 1 core(s)
26/11/04 15:04:01 INFO StandaloneSchedulerBackend: Granted executor ID app-20261104150401-0000/0 on hostPort 172.18.0.6:40761 with 1 core(s), 1024.0 MiB RAM
26/11/04 15:04:01 INFO Master: Start scheduling for app app-20261104150401-0000 with rpId: 0
26/11/04 15:04:01 INFO StandaloneAppClient$ClientEndpoint: Executor updated: app-20261104150401-0000/0 is now RUNNING
26/11/04 15:04:01 INFO StandaloneSchedulerBackend: SchedulerBackend is ready for scheduling beginning after reached minRegisteredResourcesRatio: 0.0
2026-11-04 15:04:02,948 [INFO] ✔SparkSession khởi tạo thành công.
2026-11-04 15:04:04,936 [INFO] ✔Đã kết nối tới Kafka topic 'ecommerce_transactions'.
2026-11-04 15:04:05,598 [INFO] ▶ Bắt đầu stream ghi dữ liệu vào Cassandra spark_streaming ...
2026-11-04 15:04:05,770 [INFO] 🚀 [METRICS] Query started: None (id: 1e979574-0bda-45c6-b086-c481f25f5252, runId: 3eec22c9-1bb7-42ff-acbd-dc1d578d0881)
26/11/04 15:05:38 WARN Master: Got status update for unknown executor app-20261104145014-0005/0
2026-11-04 15:06:11,164 [INFO] 💾 Ghi micro-batch #2 (1 bản ghi đã làm sạch) vào Cassandra ...
2026-11-04 15:06:28,535 [INFO] ✔Đã ghi thành công chi tiết và tổng hợp danh mục cho batch #2!
2026-11-04 15:06:28,718 [INFO] 📊 [BATCH #2] Rows: 1 | Process Rate: 0.03 rows/s | Trigger Execution: 30070 ms
2026-11-04 15:06:35,270 [INFO] 💾 Ghi micro-batch #3 (8 bản ghi đã làm sạch) vào Cassandra ...
2026-11-04 15:06:48,111 [INFO] ✔Đã ghi thành công chi tiết và tổng hợp danh mục cho batch #3!
2026-11-04 15:06:48,241 [INFO] 📊 [BATCH #3] Rows: 11 | Process Rate: 0.56 rows/s | Trigger Execution: 19554 ms
2026-11-04 15:06:54,673 [INFO] 📊 [BATCH #4] Rows: 1 | Process Rate: 0.16 rows/s | Trigger Execution: 6387 ms
```

PHẦN 3: KẾT QUẢ KIỂM THỬ VÀ ĐÁNH GIÁ

3.1. Kết quả kiểm thử các trường hợp dữ liệu

Chạy thử nghiệm gửi 13 bản ghi vào hệ thống để kiểm tra xem Spark có lọc đúng không:

STT	Trường hợp test	Dữ liệu đầu vào	Kết quả Spark xử lý	Kết quả trong Cassandra	Đánh giá
1	Đơn hàng hợp lệ	ORD-01, $803.95	Gắn nhãn HIGH_VALUE	Đã lưu đủ thông tin	**Đạt**
2	Đơn hàng rủi ro	ORD-02, $1844.26	Đánh dấu risk_flag = True	Tăng risk_count thêm 1	**Đạt**
3	Số tiền bị âm	ORD-03, -$24.36	Loại bỏ ở bước filter	Không lưu vào DB	**Đạt**
4	Thiếu email	ORD-04, email = null	Loại bỏ ở bước filter	Không lưu vào DB	**Đạt**
5	Gửi trùng đơn hàng	ORD-02 (gửi lại lần 2)	Lọc trùng bằng Watermark	Chỉ có 1 bản ghi lưu trong DB	**Đạt**

Kết luận: Sau khi thử nạp 13 dữ liệu (có 4 dữ liệu lỗi và trùng), hệ thống đã lọc sạch hoàn toàn và lưu đúng 9 bản ghi chuẩn vào Cassandra.

3.2. Đánh giá độ trễ và hiệu năng

- Tốc độ xử lý: Đạt khoảng 1.2 - 2.5 bản ghi/giây khi chạy thử trên máy cá nhân.
- Thời gian khởi chạy: Mất khoảng 13 giây ở batch đầu tiên (do phải khởi tạo kết nối Cassandra), các batch tiếp theo chạy rất nhanh chỉ từ 0.3 - 0.8 giây.
- Độ trễ trung bình: Khoảng 31.8 giây (tính từ lúc client sinh dữ liệu tới khi lưu xong vào DB).

3.3. Hạn chế

- Hạn chế: Do chạy toàn bộ 11 container trên 1 laptop cá nhân nên đôi khi bị ngốn RAM làm máy bị chậm. Nhóm cũng chưa làm được giao diện Dashboard để vẽ biểu đồ đẹp mắt mà mới chỉ xem dữ liệu qua dòng lệnh terminal.

PHẦN 4: BẢNG THEO DÕI HỆ THỐNG TRÊN LOCALHOST

Thành phần	Link Web / Câu lệnh test	Cách kiểm tra
Airflow UI	http://localhost:8080	Vào xem DAG ecommerce_multisource_streaming báo màu xanh là OK.
Kafka UI	http://localhost:8888	Vào Topic xem tin nhắn gửi về continuous.
Spark UI	http://localhost:9090	Xem status app EcommerceRealtimePipeline báo RUNNING.
Cassandra Chi tiết	docker exec cassandra cqlsh -e "SELECT * FROM spark_streaming.ecommerce_transactions LIMIT 10;"	Chạy lệnh để xem các đơn hàng đã được lọc sạch.
Cassandra Tổng hợp	docker exec cassandra cqlsh -e "SELECT * FROM spark_streaming.category_aggregates;"	Chạy lệnh để xem bảng doanh thu tổng hợp.

HƯỚNG DẪN CÁC BƯỚC CHẠY ĐỒ ÁN

Để chạy bài đồ án, mở Terminal tại thư mục PROJECT1 và gõ các lệnh sau:

PHẦN A: KHỞI TẠO HẠ TẦNG VÀ KIỂM TRA MẠNG DOCKER
Bước 1: Tạo mạng Docker chung để các container (Airflow, Kafka, Spark, Cassandra) liên lạc với nhau

docker network create airflow-kafka

Bước 2: Khởi động cụm Zookeeper, Kafka (3 brokers), Kafka-UI, Cassandra và Apache Spark Master/Worker

docker-compose -f docker-compose.yml up -d

Bước 3: Khởi động Apache Airflow (gồm Postgres Metadata và Airflow Webserver/Scheduler)

docker-compose -f docker-compose-LocalExecutor.yml up -d

Bước 4: Kiểm tra trạng thái toàn bộ container (Đảm bảo tất cả container đều ở trạng thái 'Up')

docker ps

Giao diện Web trực quan:
- Airflow Web UI: http://localhost:8080 (User: airflow / Pass: airflow)
- Kafka Web UI: http://localhost:8888 (Quản lý cụm Kafka 3 brokers, topics, messages)
- Spark Master UI: http://localhost:9090 (Quản lý cụm tính toán phân tán Spark)

PHẦN B: THỰC THI VÀ KIỂM THỬ GIAI ĐOẠN 1
Bước 5: Đợi Cassandra sẵn sàng (khoảng 30 giây) và tạo Keyspace & Table 'random_names'

docker exec -i cassandra cqlsh -e "CREATE KEYSPACE IF NOT EXISTS spark_streaming WITH replication = {'class':'SimpleStrategy','replication_factor':1}; CREATE TABLE IF NOT EXISTS spark_streaming.random_names (full_name text PRIMARY KEY, gender text, location text, city text, country text, postcode int, latitude float, longitude float, email text);"

Bước 6: Sao chép tập lệnh Spark cơ bản vào container Spark Master

docker cp spark_streaming.py spark_master:/opt/spark/spark_streaming.py
docker exec spark_master cp -v /root/.ivy2/jars/* /opt/spark/jars/

Bước 7: Chạy Spark Structured Streaming (Giai đoạn 1) để đọc từ Kafka và ghi vào Cassandra

docker exec -d -e PYTHONUNBUFFERED=1 spark_master sh -c "/opt/spark/bin/spark-submit --master spark://spark-master:7077 /opt/spark/spark_streaming.py 2>&1 | tee -a /opt/spark/spark_streaming.log > /proc/1/fd/1"

Bước 8: Chạy script lấy dữ liệu từ API randomuser.me và đẩy vào Kafka topic 'random_names'

```
docker exec airflow_webserver python /usr/local/airflow/dags/stream_to_kafka.py
# Bước 9: Kiểm tra dữ liệu thực tế đã được nạp thành công vào Cassandra bảng 'random_names'
docker exec cassandra cqlsh -e "SELECT full_name, email, city, country, postcode FROM spark_streaming.random_names LIMIT 15;"
# Bước 10: Đếm tổng số bản ghi đã ghi vào Cassandra ở Giai đoạn 1
docker exec cassandra cqlsh -e "SELECT count(*) FROM spark_streaming.random_names;"


# PHẦN C: THỰC THI VÀ KIỂM THỬ GIAI ĐOẠN 2
# Bước 11: Tạo Kafka topic mới 'ecommerce_transactions' với 3 partitions và replication factor 3
docker exec kafka1 kafka-topics --bootstrap-server kafka1:19092 --create --if-not-exists --topic ecommerce_transactions --partitions 3 --replication-factor 3
# Bước 12: Tạo Schema Cassandra nâng cao (Bảng chi tiết giao dịch và Bảng tổng hợp theo danh mục)
docker cp schema_advanced.cql cassandra:/tmp/schema_advanced.cql
docker exec cassandra cqlsh -f /tmp/schema_advanced.cql
# Bước 13: Sao chép tập lệnh Spark nâng cao vào Spark Master
docker cp spark_streaming_advanced.py spark_master:/opt/spark/spark_streaming_advanced.py
# Bước 14: Submit ứng dụng PySpark Streaming Nâng Cao (chạy nền)
docker exec -d -e PYTHONUNBUFFERED=1 spark_master sh -c "/opt/spark/bin/spark-submit --master spark://spark-master:7077 /opt/spark/spark_streaming_advanced.py 2>&1 | tee -a /opt/spark/spark_streaming_advanced.log > /proc/1/fd/1"
# Bước 15: Kích hoạt luồng thu thập dữ liệu đa nguồn từ Airflow (API Profile + Synthetic Orders)
docker exec airflow_webserver python /usr/local/airflow/dags/ecommerce_producer.py
# Bước 16: Kiểm tra bảng chi tiết giao dịch đã được làm sạch và khử trùng lặp
docker exec cassandra cqlsh -e "SELECT order_id, full_name, user_email, product_category, amount, tier, risk_flag, processing_latency_ms FROM spark_streaming.ecommerce_transactions LIMIT 20;"
# Bước 17: Kiểm tra bảng tổng hợp doanh thu và đơn hàng theo danh mục thời gian thực (Real-time Category Analytics)
```

```
docker exec cassandra cqlsh -e "SELECT product_category, total_orders, total_revenue, avg_order_value, risk_count FROM spark_streaming.category_aggregates;"
# Bước 18: Xem log đo lường hiệu năng streaming (Throughput rows/s và Latency)
docker logs spark_master --tail 30
```

PHẦN D: TÙY CHỌN DỪNG VÀ GIẢI PHÓNG TÀI NGUYÊN (CLEANUP)

Dừng hệ thống khi không sử dụng (dữ liệu vẫn được lưu trữ nguyên vẹn trên volume)
docker exec spark_master pkill -f spark_streaming_advanced.py || true
docker exec spark_master rm -rf /tmp/checkpoint_ecommerce
docker-compose -f docker-compose-LocalExecutor.yml stop
docker-compose -f docker-compose.yml stop

Khởi động lại hệ thống bất kỳ lúc nào:
docker-compose -f docker-compose.yml start
docker-compose -f docker-compose-LocalExecutor.yml start